Q1.

A $D = 5D(n/2) + \Theta(n)$ (= constant) in big O → $\log_2 5$ is at least 2 and 2>1 so $n^{\log_2 5}$

B $D = 2D(n-1) + \Theta(1)$ in big O → $2^2 D(n-2) + 2\Theta(1) \ldots 2^n D + n \cdot \Theta(1)$ just 2^n

C $D = 9D(\frac{n}{3}) + \Theta(n^2)$ in big O → $2 = \log_3 9$ → $n^2 \log n$

slowest

$n^{\log_2 5}$ vs 2^n vs $n^2 \log n$

is $\log_2 5 > 2$? Yes

$\log_2 4 = 2$ so $\log_2 5$ is at least a little > 2.

Algorithm C is the fastest which is kinda weird cause B definitely sounded the fastest at first. (Asks for Big O not Θ)

C. $C(n) = O(n^2 \log n)$

B. $B(n) = O(2^n)$

A. $A(n) = O(n^{\log_2 5})$

Q2.

$T(n) = aT(\frac{n}{b}) + O(n^d)$

$T(n) = \begin{cases} O(n^d) & d > \log_b a \\ O(n^d \log n) & d = \log_b a \\ O(n^{\log_b a}) & d < \log_b a \end{cases}$

a. $T(n) = 2T(n/3) + 1 \rightarrow 2T(n/3) + n^0$

Used MT

base dominates

$0 < \log_3 2$

$T(n) = \Theta(n^{\log_3 2})$

b. $T(n) = 5T(n/4) + n^1$

Used MT

base dominates

$1 < \log_4 5$

$T(n) = \Theta(n^{\log_4 5})$

c. $T(n) = 7T(\frac{n}{7}) + n$

$1 = \log_7 7$

$T(n) = \Theta(n \log n)$

Used MT

base and root are the same amount of work

d. $T(n) = 9T(n/3) + n^2$

$\log_3(9) = 2$

$\boxed{T(n) = \Theta(n \log n)}$

e. $T(n) = 8T(n/2) + n^3$

$\log_2(8) = 3$

$\boxed{T(n) = \Theta(n \log n)}$

MT

base and root work is equal

f. $T(n) = 49T(n/25) + n^{\frac{3}{2}} \log(n)$

disregard the $\log(n)$ for analysis

$\log_{25} 49 < \frac{3}{2}$

$\boxed{T(n) = \Theta(n^{\frac{3}{2}} \log(n))}$

g. $T(n) = T(n-1) + 2$

$T(n-1) = T(n-2) + 2$

$T(n-2) = T(n-3) + 2$

$T(n-1) = T(n-3) + 2 + 2$

$T(n) = T(n-3) + 2 + 2 + 2$

when run n times

$T(n) = T(n-k) + 2k$ where $k = n$

no base given but not necessary.

$\boxed{T(n) = \Theta(n)}$

h. $T(n) = T(n-1) + n^c$ where $c \geq 1$

$T(n-1) = T(n-2) + (n-1)^c$

$T(n-2) = T(n-3) + (n-2)^c$

$T(n-1) = T(n-3) + (n-2)^c + (n-1)^c$

$T(n) = T(n-3) + (n-2)^c + (n-1)^c + n^c$

$n = 3$ $T(0) = 0$

$T(3) = 3^c + 2^c + 1^c + 0^c$

$\sum_{i=0}^{n} i^c$

approximation by integrals.

$\int_0^n i^c \, di = \frac{n^{c+1}}{c+1}$

$c+1$ is just a constant so n^{c+1} is the answer

$\boxed{T(n) = \Theta(n^{c+1})}$

i. $T(n) = T(n-1) + c^n$ $(c > 1)$

$T(n-1) = T(n-2) + c^{n-1}$

$T(n-2) = T(n-3) + c^{n-2}$

$T(n) = T(n-3) + c^n + c^{n-1} + c^{n-2}$

$\sum_{i=0}^{n} c^i$

$\int_0^n c^i \, di \approx$

$\frac{c^n}{\ln(c)} \approx c^n$

$\boxed{T(n) = \Theta(c^n)}$

j. $T(n) = 2T(n-1) + 1$ is just

Q1 algorithm B, so it stands to be $\Theta(2^n)$

$\boxed{T(n) = \Theta(2^n)}$

Q3. 2.12

The Program is like mergesort where there are two calls

```
f(n)
 if n>1:
   print (something)
   f(n/2)
   f(n/2)
```

$T(n) = \underbrace{2T(\tfrac{n}{2})}_{\text{calls}} + \underbrace{O(1)}_{\text{print}}$

$d = 0$

$n^d = 1$

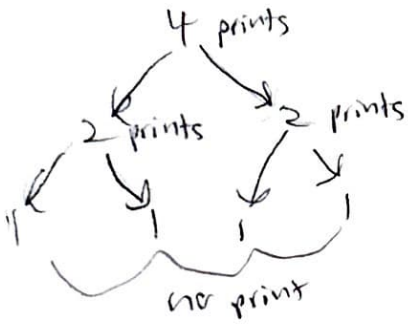
$\log_2 2 = 1$

$1 > 0$

$\Theta(n^{\log_b a}) = \Theta(n^{\log_2 2}) = \Theta(n)$

This doesn't seem right, but I followed the steps so I'm going w/ it.

num yes it does



$4 - 3 = 1$

Seems to me that it prints n-1 times

Q4. 2.16

I feel like this is just binary search but the bound "n" must be found first (in $\Theta(\log n)$ time too). Invert binary process and multiply the index by 2 until infinity or x.

First declare index at 1 and run a loop doubling index until a[index] = ∞ or x. While doubling, save the previous index each time (current index / 2).

Once you have those, return if a[index] = x.

$\text{mid} = \frac{\text{prev} + \text{index}}{2}$

Now loop until prev <= index, w/ each time returning mid if a[mid] = x, setting prev to mid+1 if a[mid] < x, or setting index to mid-1 if a[mid] > x. If mid isn't eventually returned, x is not in a. I wrote my pseudocode using iteration, but I can still use recurrent relations to show the Θ is $\Theta(\log(n))$

I'm gonna assume this is possible

```
index = 1
while (a[index] != ∞ ||
      a[index] != x);
    prev = index;
    index *= 2;
if (a[index] == x):
    return index;

while (prev <= index)
    mid = (prev + index)/2;
    if (a[mid] == x)
        return mid;
    if (a[mid] < x])
        prev = mid+1;
    else
        index = mid-1;
return err; // not found
```

code

index and prev creation : $T(n) = T(\frac{n}{2}) + \Theta(1) \rightarrow \log_2 1 = 0 \quad d = 0$

$n^d \log n = \log(n) \rightarrow \Theta(\log(n))$

binary search : $T(n) = T(\frac{n}{2}) + \Theta(1) \rightarrow \log_2 1 = 0 = d = \log(n) \rightarrow \Theta(\log(n))$

$\Theta \log(n) + \Theta \log(n) = \Theta(2\log(n)) = \boxed{\Theta(\log(n))}$

Q5. 2.19 k arrays w/ n elmnts

a. bad implementation

Suppose our k arrays of size n are configured as a 2d array so the row is k and the column is n. (array[k][n])

```
finalArray[k*n];
merge():
    for(i=0; i<k; i++)
        SortArray(finalArray, array[i])
return finalArray
```

// takes array[i] and sorts its values into finalArray in $\Theta(kn)$ time

call	time
0	n
1	2n
2	3n
⋮	⋮
k	(k+1)n

$$\sum_{i=0}^{k} (k+1)n$$

$$\int_0^k (k+1)n\,dk$$

$$\left[\frac{k^2n + kn}{2}\right]_0^k$$

$\frac{k^2n + kn}{2}$ k^2n dominates

k^2n

$T(n) = \Theta(k^2n)$

b. better, divide and conquer approach

(should get something like $kn\log(k)$ b/c the non-recursive work takes $\Theta(kn)$ time)

```
finalArray[k*n];
start = 0 // first array
end = k-1 // last array
merge(array, start, end);
    if start >= end:
        return array[end]; // base
    mid = (start + end)/2;
    merge(array, start, mid);
    merge(array, mid+1, end);
    return sortArray(); // Assume SortArray works as intended
```

Instead of merging arrays one by one, if you split the arrays into groups w/ two arrays in each and merge from there, you create the reccurence:

$T(n) = 2T(\frac{k}{2}) + \Theta(nk)$

$\log_2 2 = 1$ $k^1 = k$

$1 = 1$

$T(n) = \Theta(nk\log k)$ case 2 for masters

$nk\log k$ vs nk^2

$\log k$ vs k

$\log k$ is better than k